# TD/TP "Programmation par Contraintes et Optimisation"

LEBBAH Yahia

Le 24 octobre 2026

# Table des matières


**1 TD - Familiarisation avec la programmation par contraintes et optimisation** **1**

1.1 TD1 . . . 1

1.1.1 Modélisation : Sudoku . . . 1

1.1.2 Définitions de base des CSPs . . . 1

1.1.3 AC3 . . . 2

1.1.4 GCC . . . 2

1.1.5 AllDifferent . . . 3

1.1.6 Dynamic Backtracking . . . 3

1.1.7 Contrôle AC3 . . . 4

1.2 TD2 : Modélisation (minizinc) . . . 4

1.2.1 Problème de planification de production . . . 5

1.2.2 Problème d'affectation . . . 5

1.2.3 Ordonnancement Job-Shop . . . 5

1.2.4 Problème des séries magiques . . . 5

1.2.5 Problème sac-à-dos . . . 5

1.2.6 Problème "social golfers" . . . 5

1.2.7 Problème "nurse rostering" . . . 5

1.2.8 Problème carré latin . . . 5

1.2.9 ... . . . 5

**2 TP - Programmation par Contraintes et Optimisation** **7**

2.1 TP1 . . . 7

2.1.1 Installation de l'environnement MiniZinc `https://www.minizinc.org/` . . . 7

# Chapitre 1

# TD - Familiarisation avec la programmation par contraintes et optimisation

## 1.1 TD1

### 1.1.1 Modélisation : Sudoku

Soit le puzzle Sudoku : *"The objective is to fill a* $9 \times 9$ *grid with digits so that each column, each row, and each of the nine* $3 \times 3$ *sub-grids that compose the grid contains all of the digits from 1 to 9. The puzzle setter provides a partially completed grid, which typically has a unique solution."*

Ou encore le texte disponible sur Wikipedia : *Le but du jeu est de remplir la grille avec une série de chiffres (ou de lettres ou de symboles) tous différents, qui ne se trouvent jamais plus d'une fois sur une même ligne, dans une même colonne ou dans une même sous-grille. La plupart du temps, les symboles sont des chiffres allant de 1 à 9, les sous-grilles étant alors des carrés de 3 × 3. Quelques symboles sont déjà disposés dans la grille, ce qui autorise une résolution progressive du problème complet.*

Proposer une modélisation du jeu Sudoku sous forme d'un CSP $\langle \mathcal{X}, \mathcal{D}, \mathcal{C} \rangle$.

### 1.1.2 Définitions de base des CSPs

Soit le problème $P(n)$ : *Il s'agit de placer des nombres parmi les* $n^2$ *premiers entiers dans un carré* $n \times n$ *de telle sorte que les sommes des lignes, des colonnes et des deux grandes diagonales soient égales.*

1. Modéliser $P(3)$ sous forme d'un CSP.
2. Tracer l'hypergraphe des contraintes de $P(3)$ ;
3. Tracer l'hypergraphe de consistance de $P(2)$ en considérant le domaine de variation de toutes les variables égale à l'intervalle $[0, 1]$ ; en déduire les solutions !

4. Supposons que la somme (de toute ligne, toute colonne et toute grande diagonale) dans $P(2)$ égale à 8 ;
   (a) Etablisser pas à pas la trace de l'application de l'algorithme AC3 d'*arc*-consistance sur ce CSP.
   (b) Tracer l'arborescence de l'application de l'algorithme $BT$ — backtrack chronologique — en prenant un *bon* ordre des variables (indication : en cas d'échec, il faut préciser les contraintes qui sont responsables) ;

### 1.1.3 AC3

— Soit le CSP P :

$$
\begin{array}{l}
X = \{x_1, x_2, x_3\};\\
D = \{D_1, D_2, D_3\};\\
C = \{C_1, \ldots, C_6\};\\
D_1 = D_2 = D_3 = \{1, \ldots, 10\};\\
C_1 : 1 \leq x_1 \leq 4;\\
C_2 : 1 \leq x_2 \leq 4;\\
C_3 : 1 \leq x_3 \leq 4;\\
C_4 : x_1 + x_2 = 4;\\
C_5 : x_1 + x_3 = 5;\\
C_6 : x_2 + x_3 \geq 6;
\end{array}
$$

— (1) Tracez l'exécution de l'algorithme AC3 sur le CSP P, en montrant à chaque itération :
   — les fonctions de projection $\pi_{j,i}$ qui ont permis de réduire les domaines, en précisant la contrainte $C_j$ et la variable $x_i$.
   — les domaines réduits.

### 1.1.4 GCC

Soit le CSP P

$$
\begin{array}{l}
x \in \{0, 1, 2\}\\
y \in \{1, 2\}\\
z \in \{0, 1\}\\
c_{31} : card(2, \{x, y, z\}, [2, 2])\\
c_{32} : card(0, \{x, y, z\}, [0, 1])\\
c_{33} : card(1, \{x, y, z\}, [0, 1])
\end{array}
$$

où $card(i, X, [lb, ub])$ est la contrainte de cardinalité qui impose le nombre d'occurrence de $i$ dans les solutions affectant l'ensemble $X$ a être compris entre $lb$ et $ub$.

1. Etablir manuellement l'*arc*-consistance avec la contrainte GCC formée de $\{c_{31}, c_{32}, c_{33}\}$.

### 1.1.5 AllDifferent

Nous voulons associer quatre tâches à cinq machines. On ne peut assigner plus d'une tâche à une machine. La table ci-dessous montre les combinaisons possibles.

| Tâche | Machines |
|---|---|
| 1 | B, C, D, E |
| 2 | B, C |
| 3 | A, B, C, D |
| 4 | B, C |

Nous créons une variable par tâche, qui prend comme valeur la machine assignée. Les domaines initiaux de ces variables sont donnés dans la table ci-dessus.

1. (1.5/20) Modélisez ce problème sous forme d'un CSP en utilisant comme contraintes la contrainte globale de différence.
2. (3.5/20) Tracez l'algorithme de filtrage de la contrainte de différence.

### 1.1.6 Dynamic Backtracking

Soit le graphe de la Figure 1.1 [1]. Chacun des sommets peut être affecté d'une valeur entière appartenant à l'ensemble $\{1, 2, 3, 4, 5, 6\}$. Nous voulons affecter une valeur à chacun des sommets du graphe de sorte que la valeur de tout sommet $a$ doit être différente de celle de tout sommet $b$ adjacent à $a$, sans que les deux valeurs $a$ et $b$ soit consécutives. Formellement

$$(|a - b| > 1).$$

1. Modélisez ce problème sous forme d'un CSP.
2. Tracez l'algorithme de "dynamic backtracking" pour affecter le graphe de la Figure 1.1.

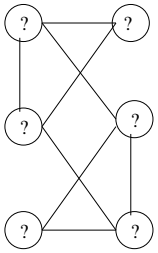



FIGURE 1.1 – Exemple d'un graphe

**Rappels sur** *dynamic backtracking* :

**Définition 1** (mécanisme d'explication)**.** *Soit $P$ une instanciation partielle localement consistante. Une explication éliminante d'une variable $x_i \in X$ est une paire $(d_i, S)$ où $d_i \in D_i$ et $S$ est un sous-enseble de $vars(P)$ l'ensemble des variables*

1. Extrait de la présentation *Peter Van Beek, Introduction to Constraint Programming, CPAIOR 2009.*

*affectées dans $P$. Un mécanisme d'élimination $EX$ d'un CSP est une fonction qui prend comme argument une instanciation partielle localement consistante $P$ et une variable $x_i$ n'appartenant pas à $vars(P)$, et retourne l'ensemble $EX(P, x_i)$ d'explications élminantes de $x_i$.*

1. soit $P = E_i = \emptyset$ pour tout $x_i \in X$ ;
2. si $vars(P) = X$, retouner la solution $P$. Sinon, sélectionner une variable $x_i \in X - vars(P)$ ; soit $E_i = E_i \cup EX(P, x_i)$ ;
3. soit $S = D_i - vals(E_i)$. Si $S$ est non vide, choisir un élément $d_i$ de $S$ ; ajouter $(x_i, d_i)$ à $P$ et revenir à l'étape (2) ;
4. si $S$ est vide, on a $vals(E_i) = D_i$ ; soit $E$ l'ensemble de toutes les variables apparaissant dans les explications des valeurs éliminées ;
5. si $E = \emptyset$, alors échec, pas de solutions. Sinon, soit $(x_j, d_j)$ la dernière entrée dans $P$ dont la variable est dans $E$ ; supprimer $(x_j, d_j)$ de $P$ et, pour chaque variable $x_k$ affectée après la variable $x_j$, supprimer de $E_k$ toutes les explications éliminantes qui necessitent $x_j$ ; ajouter $(d_j, E \cap vars(P))$ à $E_j$ et revenir à l'étape (2) ;

### 1.1.7 Contrôle AC3

Dérouler l'algorithme AC3 sur le CSP
— $\mathcal{X} = \{x_1, x_2, x_3, x_4\}$
— $\mathcal{D} = \{D_1, D_2, D_3, D_4\}$, avec $D_1 = \{a\}, D_2 = \{a, b\}, D_4 = D_3 = \{a, b, c, d\}$
— $\mathcal{C} = \{(c_1 : x_1 \neq x_2), (c_2 : x_1 \neq x_3), (c_3 : x_1 \neq x_4), (c_4 : x_2 \neq x_3), (c_5 : x_2 \neq x_4), (c_6 : x_3 \neq x_4)\}$

Rappel : Algorithme AC3

```
1: L := C ;
2: while L est non vide do
3:    Choisir et supprimer dans L une contrainte C_j(x_{j_1}, ..., x_{j_l}) ;
4:    for x_k ∈ {x_{j_1}, ..., x_{j_l}} do
5:       if π_{j,k}(D) ≠ D_k then
6:          D_k := π_{j,k}(D) ;
7:          L := L ∪ {C_p | x_k ∈ var(C_p)} ;
8:       end if
9:    end for
10: end while
```

## 1.2 TD2 : Modélisation (minizinc)

1. Une documentation détaillée sur minizinc est disponible dans les liens :
   (a) PDF : https://docs.minizinc.org/_/downloads/en/stable/pdf/
   (b) HTML : https://docs.minizinc.org/en/stable/

(c) Tutoriel très court du langage MiniZinc : https://imada.sdu.dk/u/marco/DM877/assets/dm877-lec5.pdf

(d) Tutoriel court du langage MiniZinc : https://www.cse.cuhk.edu.hk/~jlee/courses/csc5240/notes/moocNEW/mzTutorial.pdf

### 1.2.1 Problème de planification de production

Page 39. [MiniZinc Handbook Release 2.10.1]

### 1.2.2 Problème d'affectation

Page 53. [MiniZinc Handbook Release 2.10.1]

### 1.2.3 Ordonnancement Job-Shop

Page 54. [MiniZinc Handbook Release 2.10.1]

### 1.2.4 Problème des séries magiques

Page 59. [MiniZinc Handbook Release 2.10.1]

### 1.2.5 Problème sac-à-dos

Page 61. [MiniZinc Handbook Release 2.10.1]

### 1.2.6 Problème "social golfers"

Page 61. [MiniZinc Handbook Release 2.10.1]

### 1.2.7 Problème "nurse rostering"

Page 71. [MiniZinc Handbook Release 2.10.1]

### 1.2.8 Problème carré latin

Page 122. [MiniZinc Handbook Release 2.10.1]

### 1.2.9 ...

# Chapitre 2

# TP - Programmation par Contraintes et Optimisation

## 2.1 TP1

### 2.1.1 Installation de l'environnement MiniZinc https://www.minizinc.org/

(a) Télécharger l'environnement open-source MiniZinc de son site officiel https://www.minizinc.org/resources/

(b) Une documentation détaillée est disponible dans les liens :

i. PDF : https://docs.minizinc.org/_/downloads/en/stable/pdf/

ii. HTML : https://docs.minizinc.org/en/stable/

iii. Tutoriel très court du langage MiniZinc : https://imada.sdu.dk/u/marco/DM877/assets/dm877-lec5.pdf

iv. Tutoriel court du langage MiniZinc : https://www.cse.cuhk.edu.hk/~jlee/courses/csc5240/notes/moocNEW/mzTutorial.pdf

(c) MiniZinc est un langage de modélisation et un environnement exploitant de nombreux solveurs dans le domaine libre ou commercial, en particulier :

i. OR-Tools (CP/PL/SAT) : https://developers.google.com/optimization

ii. Gecode (CP) : https://www.gecode.dev/

iii. CBC (OR) : https://github.com/coin-or/Cbc

iv. ... Si installé ...

v. IBM ILOG CPLEX : https://www.ibm.com/products/ilog-cplex-optimization

vi. Gurobi (PL) : https://www.gurobi.com/

vii. SCIP (PL/CP) : https://www.scipopt.org/

viii. CHOCO (CP) : https://choco-solver.org/

(d) Lancer un nouveau model avec le contenu du problème exemple Send+More=Money :

```
include "alldifferent.mzn";
var 1..9: S;
var 0..9: E;
```

```
var 0..9: N;
var 0..9: D;
var 1..9: M;
var 0..9: O;
var 0..9: R;
var 0..9: Y;
constraint 1000 * S + 100 * E + 10 * N + D
+ 1000 * M + 100 * O + 10 * R + E
= 10000 * M + 1000 * O + 100 * N + 10 * E + Y;
constraint alldifferent([S,E,N,D,M,O,R,Y]);
solve satisfy;
  output [" \(S)\(E)\(N)\(D)\n",
    "+ \(M)\(O)\(R)\(E)\n",
    "= \(M)\(O)\(N)\(E)\(Y)\n"];
```

(e) Réécrire le modèle avec la stratégie de modélisation proposée dans le cours.

(f) Proposer une modélisation du problème des $n$-reines.